

소프트웨어공학

Homework #4

2019060546

주원웅

True_Positive

```
juwowa@juwowa: ~/clang+llvm-18.1.8-aarch64-linux-gnu/bin
#include <stdlib.h>
#include <stdio.h>

void test_leak() {
    int *ptr = (int *)malloc(10 * sizeof(int));
    ptr[0] = 42;
    // free(ptr); // 의도적으로 free 누락
}

int main() {
    test_leak();
    return 0;
}
```

1,1 All

95% 12/20, 5:47 PM gcc

```
juwowa@juwowa: ~/clang+llvm-18.1.8-aarch64-linux-gnu/bin
juwowa@juwowa:~/clang+llvm-18.1.8-aarch64-linux-gnu/bin$ scan-build clang -o output true_positive.c
scan-build: Using '/mnt/hgfs/share/clang+llvm-18.1.8-aarch64-linux-gnu/bin/clang-18' for static analysis
true_positive.c:8:1: warning: Potential leak of memory pointed to by 'ptr' [unix.Malloc]
    8 | }
      | ^
1 warning generated.
scan-build: Analysis run complete.
scan-build: 1 bug found.
scan-build: Run 'scan-view /tmp/scan-build-2024-12-20-084650-1886-1' to examine bug reports.
juwowa@juwowa:~/clang+llvm-18.1.8-aarch64-linux-gnu/bin$ gcc -fsanitize=address -g -o true_positive true_positive.c
juwowa@juwowa:~/clang+llvm-18.1.8-aarch64-linux-gnu/bin$ ./true_positive

=====
==1954==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 40 byte(s) in 1 object(s) allocated from:
#0 0xfffffae00a2f4 in __interceptor_malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:145
#1 0xaaaaabaf808e0 in test_leak /home/juwowa/clang+llvm-18.1.8-aarch64-linux-gnu/bin/true_positive.c:5
#2 0xaaaaabaf80958 in main /home/juwowa/clang+llvm-18.1.8-aarch64-linux-gnu/bin/true_positive.c:11
#3 0xfffffadd73f8 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#4 0xfffffadd74c8 in __libc_start_main_impl ../csu/libc-start.c:392
#5 0xaaaaabaf807ec in _start (/home/juwowa/clang+llvm-18.1.8-aarch64-linux-gnu/bin/true_positive+0x7ec)

SUMMARY: AddressSanitizer: 40 byte(s) leaked in 1 allocation(s).
juwowa@juwowa:~/clang+llvm-18.1.8-aarch64-linux-gnu/bin$
```

95% 12/20, 5:47 PM gcc

True positive 를 보여주는 코드이다. 동적할당 메모리를 Free 하지 않았기에 memory leak 가 발생해야 한다.

Static analysis 실행 시 memory leak 을 잘 잡아내고 실제 실행 시 에러 재현도 잘 된다.

False_Positive

```
juwowa@juwowa: /tmp/scan-build-2024-12-20-075627-1612-1
juwowa@juwowa:~/clang+llvm-18.1.8-aarch64-linux-gnu/bin$ scan-build clang -o output b
pt.c
scan-build: Using '/mnt/hgfs/share/clang+llvm-18.1.8-aarch64-linux-gnu/bin/clang-18'
for static analysis
bpt.c:322:13: warning: Value stored to 'cpu_time_used' is never read [deadcode.DeadSt
ores]
 322 |         cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
      |         ^~~~~~
bpt.c:582:17: warning: Value stored to 'flag' is never read [deadcode.DeadStores]
 582 |         flag = 1;
      |         ^~
bpt.c:788:13: warning: Value stored to 'flag' is never read [deadcode.DeadStores]
 788 |         flag = 1;
      |         ^~
bpt.c:846:13: warning: Value stored to 'flag' is never read [deadcode.DeadStores]
 846 |         flag = 1;
      |         ^~
bpt.c:1040:9: warning: Value stored to 'cur_pos' is never read [deadcode.DeadStores]
1040 |         cur_pos = 0;
      |         ^~
bpt.c:1080:21: warning: Value stored to 'cur_pos' is never read [deadcode.DeadStores]
1080 |         cur_pos = i;
      |         ^~
bpt.c:1088:17: warning: Value stored to 'cur_pos' is never read [deadcode.DeadStores]
1088 |         cur_pos = cur->num_of_keys;
      |         ^~~~~~
bpt.c:1109:9: warning: Value stored to 'cur_pos' is never read [deadcode.DeadStores]
1109 |         cur_pos = 0;
      |         ^~
bpt.c:1135:21: warning: Value stored to 'cur_pos' is never read [deadcode.DeadStores]
1135 |         cur_pos = i;
      |         ^~
bpt.c:1143:17: warning: Value stored to 'cur_pos' is never read [deadcode.DeadStores]
1143 |         cur_pos = cur->num_of_keys;
      |         ^~~~~~
10 warnings generated.
scan-build: Analysis run complete.
scan-build: 9 bugs found.
scan-build: Run 'scan-view /tmp/scan-build-2024-12-20-075627-1612-1' to examine bug r
eports.
100% 12/20, 5:01PM export
```

분석 진행할 소스코드로 이번 학기 과제로 사용한 b+tree 구현 코드를 사용했다.

분석 시 never read 에러가 발생하지만, 실제 동작 시 모두 필요한 변수이기에 false_positive 라고 판단하였다.

False_Negative

```
juwowa@juwowa: ~/clang+llvm-18.1.8-aarch64-linux-gnu/bin

#include <stdlib.h>
#include <stdio.h>

void forget_to_free() {
    int *data = NULL;
    // free(data); // 메모리 해제 누락
    int j = 10;
    while(j > 0){
        data = (int*)malloc(sizeof(int) *5);
        j--;
    }
    free(data);
}

int main() {
    int i;
    scanf("%d",&i);
    if(i > 0)
    {
        forget_to_free();
    }
    return 0;
}
```

```
juwowa@juwowa: ~/clang+llvm-18.1.8-aarch64-linux-gnu/bin

juwowa@juwowa:~/clang+llvm-18.1.8-aarch64-linux-gnu/bin$ vi false_negative.c
juwowa@juwowa:~/clang+llvm-18.1.8-aarch64-linux-gnu/bin$ scan-build clang -o output false_negative.c
scan-build: Using '/mnt/hgfs/share/clang+llvm-18.1.8-aarch64-linux-gnu/bin/clang-18'
for static analysis
scan-build: Analysis run complete.
scan-build: Removing directory '/tmp/scan-build-2024-12-20-084527-1862-1' because it
contains no reports.
scan-build: No bugs found.
juwowa@juwowa:~/clang+llvm-18.1.8-aarch64-linux-gnu/bin$ gcc -fsanitize=address -g -o
false_negative false_negative.c
juwowa@juwowa:~/clang+llvm-18.1.8-aarch64-linux-gnu/bin$ ./false_negative
1

=====
==1879==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 180 byte(s) in 9 object(s) allocated from:
#0 0xffff911ba2f4 in __interceptor_malloc ../../../../src/libsanitizer/asan/asan_
malloc_linux.cpp:145
#1 0xaaaaafe40bf0 in forget_to_free /home/juwowa/clang+llvm-18.1.8-aarch64-linux-
gnu/bin/false_negative.c:9
#2 0xaaaaafe40d2c in main /home/juwowa/clang+llvm-18.1.8-aarch64-linux-gnu/bin/fa
lse_negative.c:20
#3 0xffff90f873f8 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.
h:58
#4 0xffff90f874c8 in __libc_start_main_impl ../csu/libc-start.c:392
#5 0xaaaaafe40aec in _start (/home/juwowa/clang+llvm-18.1.8-aarch64-linux-gnu/bin
/false_negative+0xaec)

SUMMARY: AddressSanitizer: 180 byte(s) leaked in 9 allocation(s).
juwowa@juwowa:~/clang+llvm-18.1.8-aarch64-linux-gnu/bin$
```

메모리 누수를 탐지하지 못하는 false_negative 예시 이다. Scan-build 를 통해 검사 시 bug 를 찾지 못하지만, 실제 코드 실행시 메모리 누수가 발생한다.

이유는 동적할당 된 코드를 다시 동적할당을 통해 다른 메모리블록 할당 시 원래 할당 된 메모리블록을 free 해주지 않았기 때문이다.

재현 인풋은 0이상의 정수를 입력시 재현된다.